

TECHNICIEN SPÉCIALISÉ EN DEVELOPPEMENT INFORMATIQUE

RAPPORT DE PROJET DE FIN DE FORMATION

THEME

CONCEPTION ET DÉVELOPPEMENT D'UN DASHBOARD ANALYTIQUE POUR LA DIRECTION D'UNE ÉCOLE DE TECHNICIENS SPÉCIALISÉS

Réalisé par :

- EL Mehdi MOUSSA-FARES
- Hamza BOUSSEAM
- Aya FALGHASS

Encadré par :

- Mr. Khalid AIT BEN HAMOU

Année de formation : 2025-2026

Dédicace

Nous dédions ce travail à toutes les personnes qui nous ont soutenues tout au long de ce parcours.

À **nos familles**, pour leur amour inconditionnel, leur patience et leurs encouragements permanents. Leur présence à nos côtés a été une source de force et de motivation.

À **nos enseignants**, pour leur encadrement, leur savoir transmis avec passion et leur disponibilité.

À tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet, nous exprimons notre plus profonde gratitude.

Remerciements

Nous tenons à exprimer notre sincère gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin de formation.

Nos remerciements les plus profonds vont à **notre encadrant**, pour ses conseils avisés, son suivi constant et son soutien technique tout au long de ce projet.

Nous remercions également **l'ensemble du corps professoral de l'École Racine**, pour la qualité de l'enseignement reçu durant ces deux années de formation, ainsi que pour leur engagement et leur disponibilité.

Un grand merci à **nos familles**, pour leur soutien moral et leur encouragement quotidien, sans lesquels ce travail n'aurait pas pu aboutir.

Enfin, merci à tous nos camarades de classe et amis qui ont partagé avec nous ce parcours et offert leur aide précieuse lorsque cela était nécessaire.

Résumé

Ce projet porte sur la conception et la réalisation d'un tableau de bord analytique destiné à la direction d'une école marocaine de techniciens spécialisés. Il permet d'analyser les étudiants, les modules, les inscriptions pédagogiques, les présences et les paiements des spécialités Développement Informatique et Financier Comptable.

L'application repose sur une API FastAPI, SQLAlchemy et PostgreSQL, ainsi que sur une interface React construite avec Vite et Recharts. L'authentification JWT protège les routes et distingue notamment les opérations d'administration des comptes utilisateurs.

Le tableau de bord présente des KPI, des graphiques comparatifs et des filtres par période, mois académique, spécialité, module et classe. Les résultats filtrés peuvent être exportés aux formats PDF et Excel.

Les enregistrements couvrant septembre 2025 à juillet 2026, ainsi que le montant mensuel de 1 300 MAD, constituent exclusivement des données simulées de démonstration utilisées pour tester les calculs, tendances et fonctions analytiques.

Mots-clés :

Dashboard analytique, Business Intelligence, Python, React, PostgreSQL, Éducation, Visualisation des données.

Abstract

This project concerns the design and implementation of an analytical dashboard for the management of a Moroccan specialized technician school. It analyses students, modules, enrolments, attendance and payments for Software Development and Financial Accounting.

The application uses FastAPI, SQLAlchemy and PostgreSQL for the backend and data layer, with a React frontend built with Vite and Recharts. JWT authentication protects API routes and administrative user-account operations.

The dashboard provides KPI cards, comparative charts and filters by date, academic month, specialty, module and class. Filtered reports can be exported to PDF and Excel.

Records dated from September 2025 to July 2026 and the monthly amount of MAD 1,300 are simulated demonstration data used only to test calculations, trends and analytical features.

Keywords:

Analytical Dashboard, Business Intelligence, Education Management, Data Visualization, Python, React, PostgreSQL.

Table des matières

DEDICACE	1
REMERCIEMENTS	2
RESUME	3
ABSTRACT	4
LISTE DES FIGURES	6
LISTE DES TABLEAUX	7
INTRODUCTION GENERALE	8
1. CONTEXTE ET PRESENTATION DU SUJET	8
2. OBJECTIFS DU PROJET	8
3. OBJECTIFS GENERAUX	9
4. OBJECTIFS SPECIFIQUES	9
5. METHODOLOGIE ADOPTEE	10
6. PLAN DU RAPPORT	10
CHAPITRE 1 : PRESENTATION GENERALE DU PROJET	12
1.1 CONTEXTE ET PROBLEMATIQUE	12
1.2 PRESENTATION DU DOMAINE D'APPLICATION.....	12
1.3 JUSTIFICATION DU PROJET	12
1.4 PERIMETRE FONCTIONNEL.....	13
1.5 OBJECTIFS DU CHAPITRE	13
1.6 CAHIER DES CHARGES.....	13
<i>Objectifs détaillés</i>	13
<i>Contraintes techniques et fonctionnelles</i>	13
<i>Contraintes techniques :</i>	13
<i>Contraintes fonctionnelles :</i>	13
1.7 TECHNOLOGIES CHOISIES	14
1.8 ÉTUDE DE L'EXISTANT.....	14
<i>Solutions existantes</i>	14
<i>Limites des solutions existantes</i>	14
<i>Comparaison avec la solution réalisée</i>	14
1.9 CONCLUSION DU CHAPITRE	14
CHAPITRE 2 : ANALYSE DES BESOINS	15
2.1 PRESENTATION DE L'ANALYSE	15
2.2 BESOINS FONCTIONNELS.....	15
2.3 BESOINS NON FONCTIONNELS	15
2.4 ACTEURS DU SYSTEME.....	16
2.5 BESOINS D'INFORMATION	16
2.6 SYNTHÈSE DES BESOINS	16
2.7 DIAGRAMME DE CAS D'UTILISATION.....	17
2.8 DIAGRAMME DE CLASSES	18
2.9 DIAGRAMME DE SEQUENCE.....	18
2.10 SEQUENCE DE GENERATION DES EXPORTS.....	19
2.11 SYNTHÈSE DES INTERACTIONS	20
CHAPITRE 3 : CONCEPTION ET ARCHITECTURE	21
3.1 ARCHITECTURE GENERALE	21
3.2 SCHEMA GLOBAL DU SYSTEME	21

3.3 AVANTAGES DE L'ARCHITECTURE	21
3.4 CONCLUSION DU CHAPITRE	21
CHAPITRE 4 : REALISATION DE L'APPLICATION	22
4.1 DEVELOPPEMENT DU BACKEND	22
4.2 DEVELOPPEMENT DU FRONTEND	22
4.3 CAPTURES D'ECRAN A COMPLETER.....	22
4.4 MODELISATION DE LA BASE DE DONNEES	39
4.5 FONCTIONNALITES PRINCIPALES	40
4.6 SECURITE DU SYSTEME	40
4.7 PERFORMANCE ET OPTIMISATION	41
4.8 DEPLOIEMENT LOCAL ET PERSPECTIVES DE MISE EN PRODUCTION	41
4.8.1 État actuel de l'hébergement	41
4.8.2 Étapes de mise en production.....	41
4.8.3 Solutions d'hébergement envisagées	42
4.8.4 Optimisation	42
4.8.5 Configuration de production	42
4.9 CONCLUSION DU CHAPITRE	42
CHAPITRE 5 : TESTS ET RESULTATS	43
5.1 SCENARIOS DE VALIDATION	43
5.2 RESULTATS OBTENUS	43
<i>Synthèse des réalisations.....</i>	<i>43</i>
5.3 DIFFICULTES RENCONTREES ET SOLUTIONS APORTEES.....	44
5.4 LIMITES ET AMELIORATIONS POSSIBLES	44
<i>Améliorations futures.....</i>	<i>44</i>
<i>Perspectives d'évolution</i>	<i>44</i>
CONCLUSION GENERALE	46
BIBLIOGRAPHIE & WEBOGRAPHIE GENERALE	47
BIBLIOGRAPHIE.....	47
WEBOGRAPHIE.....	47
ANNEXES.....	48

Liste des figures

Figure 1 : Diagramme de cas d'utilisation du système.....	17
Figure 2 : Diagramme de classes des modèles de données	18
Figure 3 : Séquence d'authentification JWT et de chargement des KPI	19
Figure 4 : Séquence de génération des exports PDF et Excel	20
Figure 5 : Initialisation de l'application FastAPI	23
Figure 6 : Routes d'authentification et de gestion des comptes	26
Figure 7 : Validation du jeton JWT	27
Figure 8 : Calcul des KPI dans le backend	28
Figure 9 : Modèles SQLAlchemy de la base de données	30
Figure 10 : Service API et transmission du jeton	30
Figure 11 : Chargement des KPI dans le dashboard React	31
Figure 12 : Fonction d'export dans le frontend.....	32
Figure 13 : Page de connexion	33

Figure 14 : Tableau de bord analytique.....	34
Figure 15 : Page d’analyses comparatives.....	35
Figure 16 : Gestion des comptes par le rôle Admin	36
Figure 17 : Modification du mot de passe.....	36
Figure 18 : Documentation interactive FastAPI.....	37
Figure 19 : Résultat des exports PDF et Excel	39

Liste des tableaux

Tableau 1 :acteurs du système.....	16
Tableau 2 : Scénarios de validation fonctionnelle.....	43
Tableau 3 :difficultés et solutions	44

Introduction générale

1. Contexte et présentation du sujet

Dans le cadre de la transformation numérique de la formation professionnelle, la direction doit exploiter de façon fiable les données d'effectif, de présence et de paiement. Le projet concerne une école marocaine de techniciens spécialisés.

L'objectif est de centraliser ces données dans un dashboard analytique comparant les spécialités Développement Informatique et Financier Comptable.

L'application affiche des KPI et graphiques filtrables, fournit des analyses explicables et exporte les résultats en PDF et Excel.

Le système repose sur une architecture moderne basée sur :

- Un backend en Python pour le traitement et la gestion des données.
- Un frontend interactif pour l'affichage des tableaux de bord.
- Une base de données PostgreSQL pour le stockage structuré des informations.

Ce projet vise donc à fournir une solution complète, fiable et évolutive pour améliorer la gestion des données scolaires

2. Objectifs du projet

L'objectif général est de fournir un outil de pilotage centralisé, interactif et sécurisé. Les données proviennent des tables Students, Courses, Enrollments, Payments et Attendance ; les comptes applicatifs sont gérés séparément dans la table Users.

Plus précisément, ce projet vise à :

- Centraliser les données relatives aux étudiants, inscriptions, présences et paiements dans un système unique.
- Fournir des indicateurs de performance (KPI) pour faciliter le suivi des activités de l'établissement.

- Développer des tableaux de bord interactifs avec des graphiques dynamiques et des filtres.
- Améliorer la prise de décision en offrant une vision globale et en temps réel des données.
- Automatiser l'analyse des données afin de réduire le travail manuel et les erreurs.
- Permettre l'export des résultats sous forme de rapports PDF et Excel (.xlsx).

Ce projet contribue également à la modernisation du système de gestion de l'établissement en introduisant des outils de data visualisation et d'analyse avancée.

3. Objectifs généraux

Les objectifs généraux de ce projet s'inscrivent dans une démarche de digitalisation et d'amélioration de la gestion des données au sein de l'établissement.

Ils consistent à :

- Digitaliser le système de gestion des données scolaires afin de remplacer les méthodes manuelles ou dispersées.
- Mettre en place un système centralisé permettant de regrouper toutes les informations dans une seule plateforme.
- Améliorer la qualité et la rapidité de la prise de décision grâce à des indicateurs fiables et mis à jour.
- Faciliter l'accès aux informations pour la direction à travers un tableau de bord simple et interactif.
- Optimiser l'exploitation des données en transformant les données brutes en informations utiles.
- Moderniser la gestion administrative en introduisant des outils de visualisation et d'analyse des données.

4. Objectifs spécifiques

Les objectifs spécifiques de ce projet sont définis afin de répondre de manière précise aux besoins fonctionnels du système de dashboard analytique.

Ils consistent à :

- Mettre à disposition une consultation analytique des étudiants, des inscriptions, des présences et des paiements ; la création et la suppression concernent uniquement les comptes utilisateurs administrés.
- Assurer le suivi des inscriptions et des formations au sein de l'établissement.

- Développer des indicateurs de performance (KPI) tels que le nombre d'étudiants, le taux de présence et les statistiques globales.
- Concevoir des tableaux de bord interactifs avec des graphiques clairs et dynamiques.
- Permettre le filtrage des données selon différents critères (date, classe, formation, etc.).
- Permettre l'export du rapport filtré aux formats PDF et Excel (.xlsx).
- Assurer la connexion sécurisée entre le frontend, le backend et la base de données.
- Garantir une interface simple, intuitive et facile à utiliser pour la direction.

5. Méthodologie adoptée

Pour réaliser ce projet, une méthodologie structurée a été adoptée afin d'assurer une bonne organisation du travail et un développement progressif du système.

Le projet a été réalisé selon les étapes suivantes :

- **Analyse des besoins** : identification des exigences fonctionnelles et non fonctionnelles du système, ainsi que les attentes des utilisateurs (direction).
- **Conception du système** : définition de l'architecture globale, modélisation des données et préparation des diagrammes UML.
- **Développement** : implémentation du backend en Python, développement du frontend en React et mise en place de la base de données PostgreSQL.
- **Intégration** : connexion entre le frontend, le backend et la base de données afin d'assurer le bon fonctionnement du système.
- **Tests et validation** : vérification du bon fonctionnement des différentes fonctionnalités (API, interface, affichage des statistiques).
- **Amélioration et optimisation** : correction des erreurs, optimisation des performances et amélioration de l'interface utilisateur.

Cette approche permet de garantir un développement progressif, structuré et conforme aux objectifs du projet.

6. Plan du rapport

Ce rapport est structuré en plusieurs parties afin de présenter de manière claire et progressive l'ensemble du travail réalisé dans le cadre du projet.

- Chapitre 1 – Présentation générale du projet : contexte de l'établissement, problématique, objectifs et démarche suivie.
- Chapitre 2 – Analyse des besoins : acteurs, besoins fonctionnels et non fonctionnels, règles de gestion et cas d'utilisation.
- Chapitre 3 – Conception et architecture : architecture générale, modèle de données, sécurité et choix techniques.
- Chapitre 4 – Réalisation : environnement de développement, API FastAPI, interface React, tableaux de bord, rapports et fonctions analytiques.
- Chapitre 5 – Tests et résultats : scénarios de validation, résultats obtenus, limites et améliorations possibles.
- Conclusion générale – Bilan, limites et perspectives d'évolution.

Chapitre 1 : Présentation générale du projet

1.1 Contexte et problématique

Le projet concerne une école marocaine de techniciens spécialisés proposant deux filières : Développement Informatique et Financier Comptable. Chaque filière comprend deux classes, soit quatre classes au total, et possède ses propres modules.

Dans le cadre des données de démonstration utilisées pour tester l'application, les inscriptions sont simulées à partir de septembre 2025. Le scénario de test applique un montant mensuel simulé de 1 300 MAD pendant dix mois, avec des retards et des règlements cumulés. Ces valeurs ne constituent ni des archives réelles ni un tarif officiel.

Problématique :

Comment centraliser et exploiter efficacement les données scolaires afin de permettre une analyse rapide, fiable et aider à la prise de décision ?

1.2 Présentation du domaine d'application

L'application vise à transformer ces données opérationnelles en indicateurs fiables : effectifs, recettes encaissées et attendues, assiduité, absences, comparaison des spécialités, modules problématiques et étudiants à risque.

L'objectif est de centraliser ces données dans une seule plateforme afin de les transformer en informations utiles pour l'analyse et la prise de décision.

1.3 Justification du projet

La mise en place d'un dashboard analytique est nécessaire pour plusieurs raisons :

- Les données scolaires sont dispersées et difficiles à exploiter manuellement.
- La direction manque d'outils de visualisation clairs et interactifs.
- Le suivi des performances est souvent lent et non automatisé.
- Les décisions sont prises sans analyse approfondie des données.

Ainsi, ce projet permet de moderniser la gestion scolaire et d'améliorer la qualité de l'analyse des données.

1.4 Périmètre fonctionnel

Le système doit répondre aux besoins suivants :

Gestion des comptes utilisateurs

1.5 Objectifs du chapitre

Ce chapitre permet de :

- Comprendre le contexte général du projet
 - Identifier la problématique principale
 - Définir les besoins du système
-

1.6 Cahier des charges

Objectifs détaillés

Ce projet vise à développer un dashboard analytique pour la direction permettant de centraliser et analyser les données scolaires.

Les objectifs sont :

- Centralisation des données scolaires
 - Création de tableaux de bord interactifs
 - Visualisation des KPI en temps réel
 - Automatisation des analyses
 - Aide à la prise de décision
-

Contraintes techniques et fonctionnelles

Contraintes techniques :

- Utilisation de technologies modernes (FastAPI, React, PostgreSQL)
- Communication via API REST
- Sécurité des données
- Bonne performance même avec grand volume de données

Contraintes fonctionnelles :

- Interface simple et intuitive
 - Affichage rapide des statistiques
 - Gestion fiable des données
-

1.7 Technologies choisies

Les technologies utilisées sont :

- Backend : FastAPI (Python) pour les API REST
- Frontend : React, Vite, Recharts et Lucide React
- Base de données : PostgreSQL avec SQLAlchemy
- Outils : Git, VS Code, pgAdmin et Postman

1.8 Étude de l'existant

Avant de proposer notre solution, une étude des systèmes existants a été réalisée.

Solutions existantes

- Utilisation de fichiers Excel
- Gestion manuelle des données
- Outils simples de reporting

Limites des solutions existantes

- Manque de centralisation des données
- Difficulté d'analyse rapide
- Absence de dashboards interactifs
- Risque d'erreurs humaines

Comparaison avec la solution réalisée

Notre solution propose plusieurs améliorations :

- Centralisation des données dans PostgreSQL
- Dashboard interactif avec KPI calculés à la demande
- Analyses explicables
- Interface en français
- Export PDF et Excel

1.9 Conclusion du chapitre

Ce chapitre a permis de présenter le contexte du projet, d'identifier la problématique, de définir les besoins et de comparer les solutions existantes.

Il constitue la base de conception du système de dashboard analytique.

Chapitre 2 : Analyse des besoins

2.1 Présentation de l'analyse

L'analyse des besoins permet d'identifier les fonctionnalités et les contraintes nécessaires au bon fonctionnement du système de dashboard analytique destiné à la direction.

2.2 Besoins fonctionnels

Le système doit permettre de répondre aux besoins suivants :

- Consultation analytique des étudiants ; la création et la suppression concernent uniquement les comptes utilisateurs administrés.
- Gestion des inscriptions et des formations.
- Suivi des présences et des absences au niveau de chaque module.
- Visualisation des données sous forme de tableaux de bord interactifs.
- Affichage des indicateurs de performance (KPI).
- Filtrage des données selon différents critères (date, classe, formation...).
- Export des rapports filtrés aux formats PDF et Excel.
- Authentification, consultation de la liste des utilisateurs, changement de mot de passe et gestion des comptes selon le rôle.

2.3 Besoins non fonctionnels

Ces besoins définissent les qualités que doit respecter le système :

- **Performance** : le système doit répondre rapidement aux requêtes et recalculer les statistiques selon les filtres.
- **Sécurité** : protection des données et accès sécurisé via authentification.
- **Fiabilité** : garantir la cohérence et l'exactitude des données.
- **Ergonomie** : interface simple, intuitive et adaptée aux rôles Admin et Personnel.
- **Maintenabilité** : code structuré et facilement modifiable.
- **Évolutivité** : possibilité d'ajouter de nouvelles fonctionnalités à l'avenir.

2.4 Acteurs du système

Le système distingue deux rôles applicatifs : Admin et Personnel. Les deux rôles peuvent accéder aux fonctionnalités analytiques principales. La différence principale concerne la gestion avancée des comptes utilisateurs, car la création et la suppression des utilisateurs sont réservées au rôle Admin.

- Admin : accède au tableau de bord, consulte les KPI et les analyses, applique les filtres, exporte les rapports, consulte la liste des utilisateurs, crée et supprime les comptes utilisateurs selon les règles de sécurité, et modifie son mot de passe.
- Personnel : accède au tableau de bord, consulte les KPI et les analyses, applique les filtres, exporte les rapports, consulte la liste des utilisateurs et modifie son mot de passe. Il ne peut pas créer ni supprimer les utilisateurs.

Tableau 1 :acteurs du système

Rôle	Permissions
Admin	Accès aux fonctions analytiques, à la liste des utilisateurs, à la création et à la suppression des comptes, ainsi qu'au changement de mot de passe.
Personnel	Accès aux fonctions analytiques, aux exports, à la liste des utilisateurs et au changement de mot de passe, sans création ni suppression de comptes.

2.5 Besoins d'information

Le système doit fournir des informations claires et exploitables :

- Nombre total d'étudiants.
- Taux de présence et d'absence.
- Statistiques des inscriptions par période.
- Suivi des paiements.
- Analyse des performances globales de l'établissement.

2.6 Synthèse des besoins

Cette analyse permet de définir clairement les attentes du système et de guider les étapes de conception et de développement. Elle constitue une base essentielle pour la réalisation du dashboard analytique.

2.7 Diagramme de cas d'utilisation

La figure suivante présente le diagramme de cas d'utilisation du tableau de bord analytique.

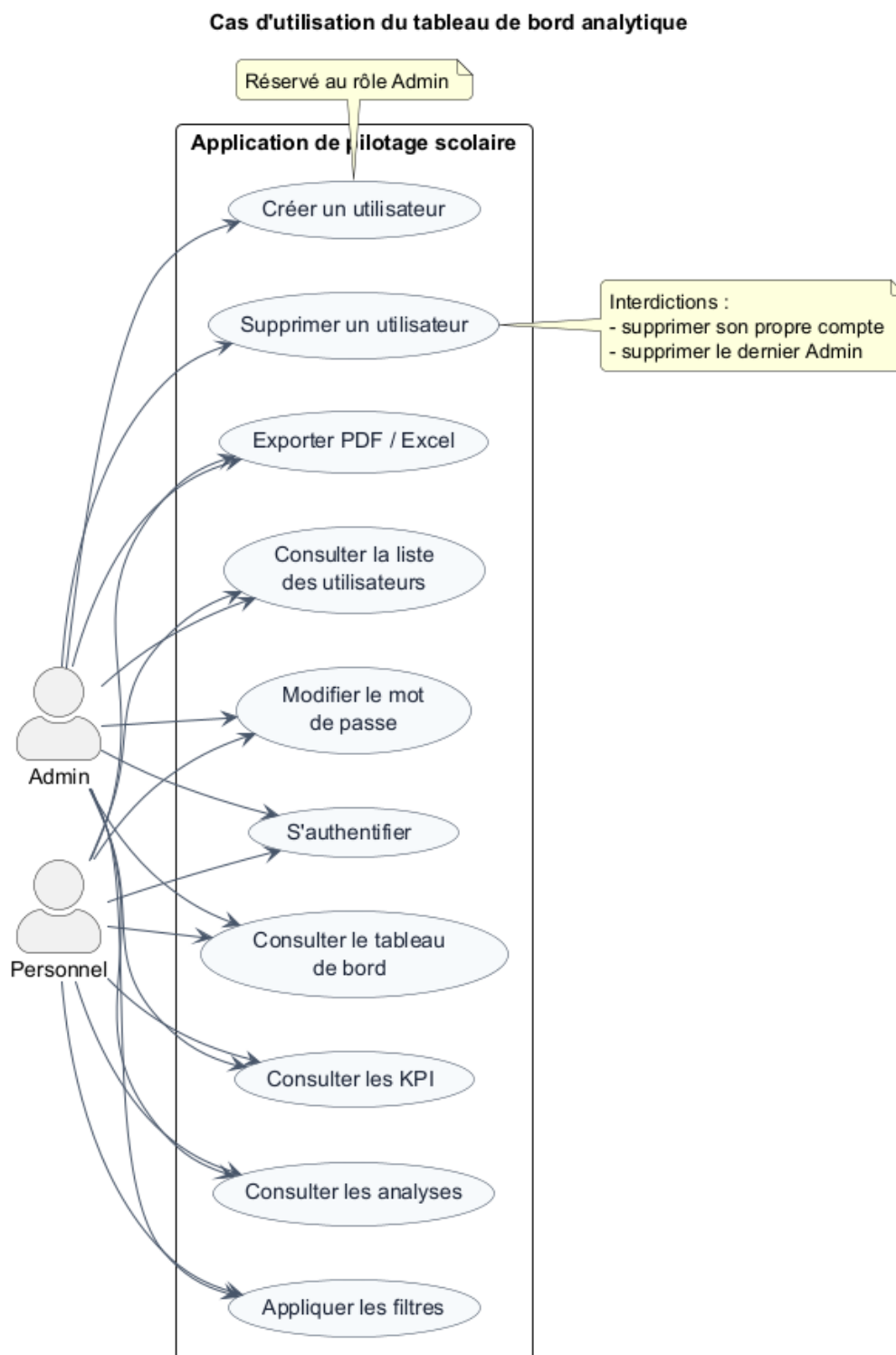


Figure 1 : Diagramme de cas d'utilisation du système

Ce diagramme présente les deux rôles applicatifs du système. Les rôles Admin et Personnel accèdent aux fonctionnalités analytiques principales, tandis que la création et la suppression des comptes utilisateurs sont réservées au rôle Admin.

2.8 Diagramme de classes

Le diagramme de classes doit représenter les modèles SQLAlchemy réellement persistés : Student, Course, Enrollment, Payment, Attendance et User.

Student est rattaché à une spécialité et à une classe.

Course représente un module associé à une spécialité.

Enrollment relie un étudiant à un module et conserve la date d'inscription pédagogique.

Payment contient l'identifiant de l'étudiant, le montant et la date du règlement. Dans models.py, student_id est une référence logique sans contrainte ForeignKey déclarée.

Attendance contient les identifiants de l'étudiant et du module, le statut et la date. Dans models.py, student_id et course_id sont des références logiques sans contraintes ForeignKey déclarées.

User stocke l'e-mail unique, le nom complet, le mot de passe haché, le rôle admin ou personnel, l'état actif et la date de création.

La figure suivante présente le diagramme de classes correspondant aux modèles SQLAlchemy.

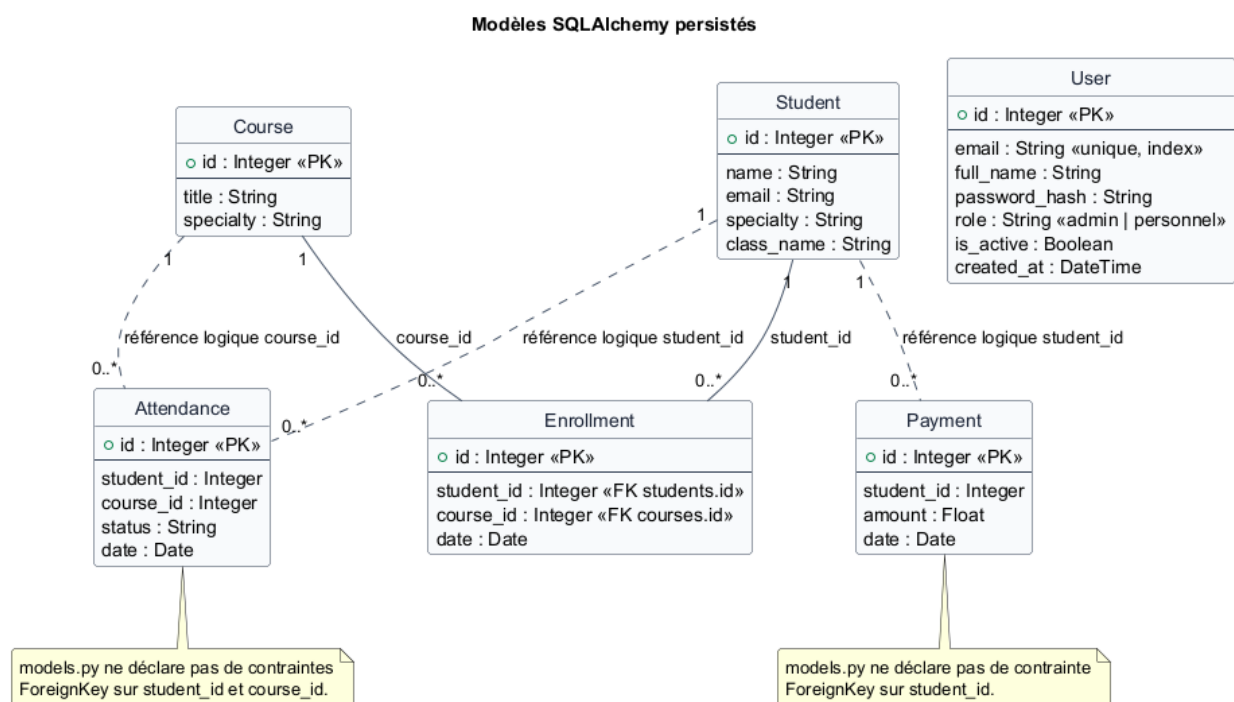


Figure 2 : Diagramme de classes des modèles de données

Ce diagramme présente les modèles persistés dans la base de données PostgreSQL ainsi que les relations utilisées pour calculer les indicateurs du tableau de bord.

2.9 Diagramme de séquence

Diagramme de séquence : consultation du tableau de bord analytique

Acteurs : Admin ou Personnel → Interface React → API FastAPI → Dépendance JWT → SQLAlchemy → Base de données PostgreSQL

Étapes :

1. L'utilisateur Admin ou Personnel se connecte à l'application.
2. **Le frontend envoie une requête API au backend.**
3. **Le backend récupère les données depuis PostgreSQL.**
4. **Les statistiques et KPI sont générés.**
5. Les graphiques et indicateurs sont affichés dans le tableau de bord.

La figure suivante présente la séquence d'authentification et de chargement des KPI.

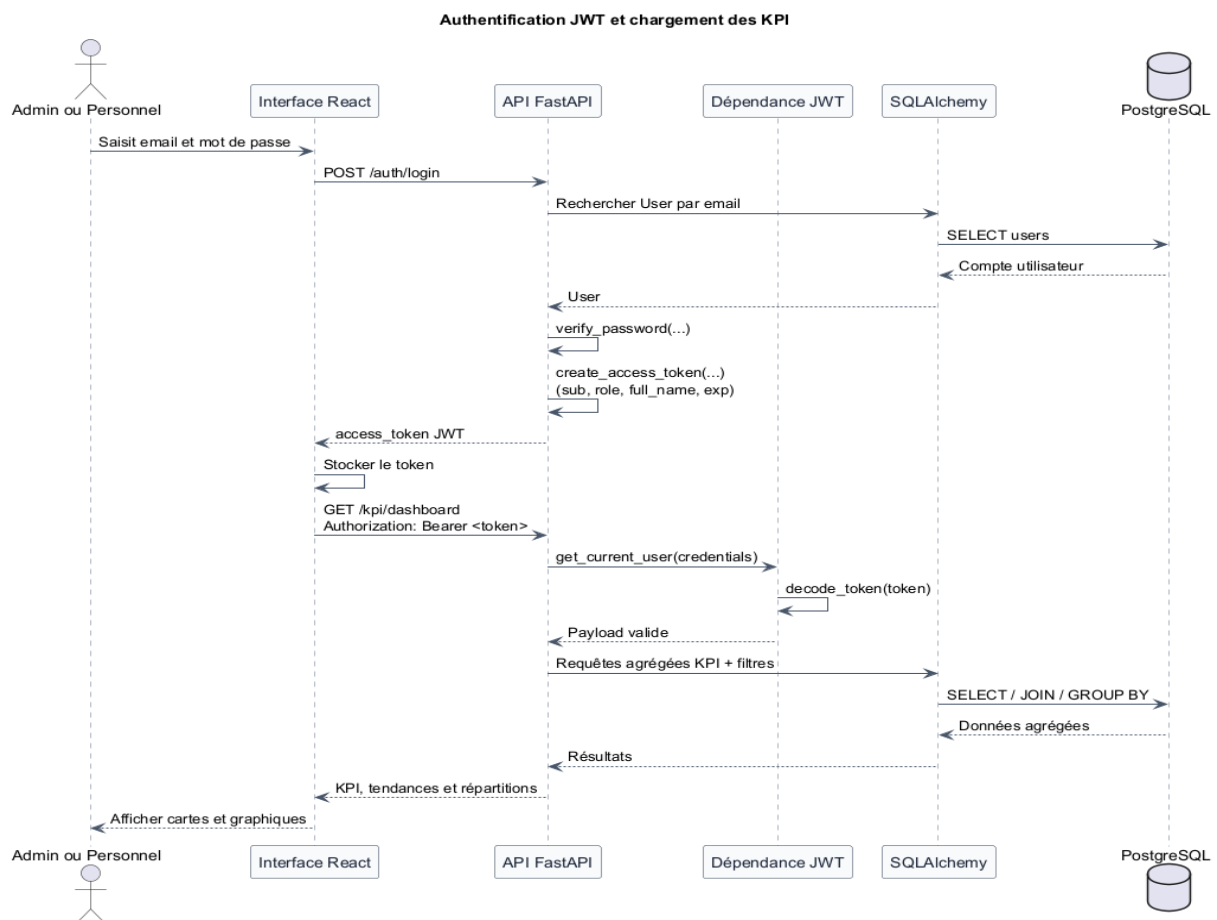


Figure 3 : Séquence d'authentification JWT et de chargement des KPI

Cette séquence décrit la connexion d'un utilisateur Admin ou Personnel, la création du jeton JWT, sa validation sur une route protégée et le chargement des KPI depuis PostgreSQL via SQLAlchemy.

2.10 Séquence de génération des exports

La route protégée /kpi/export/dashboard génère un fichier PDF ou XLSX après application des mêmes filtres que le tableau de bord.

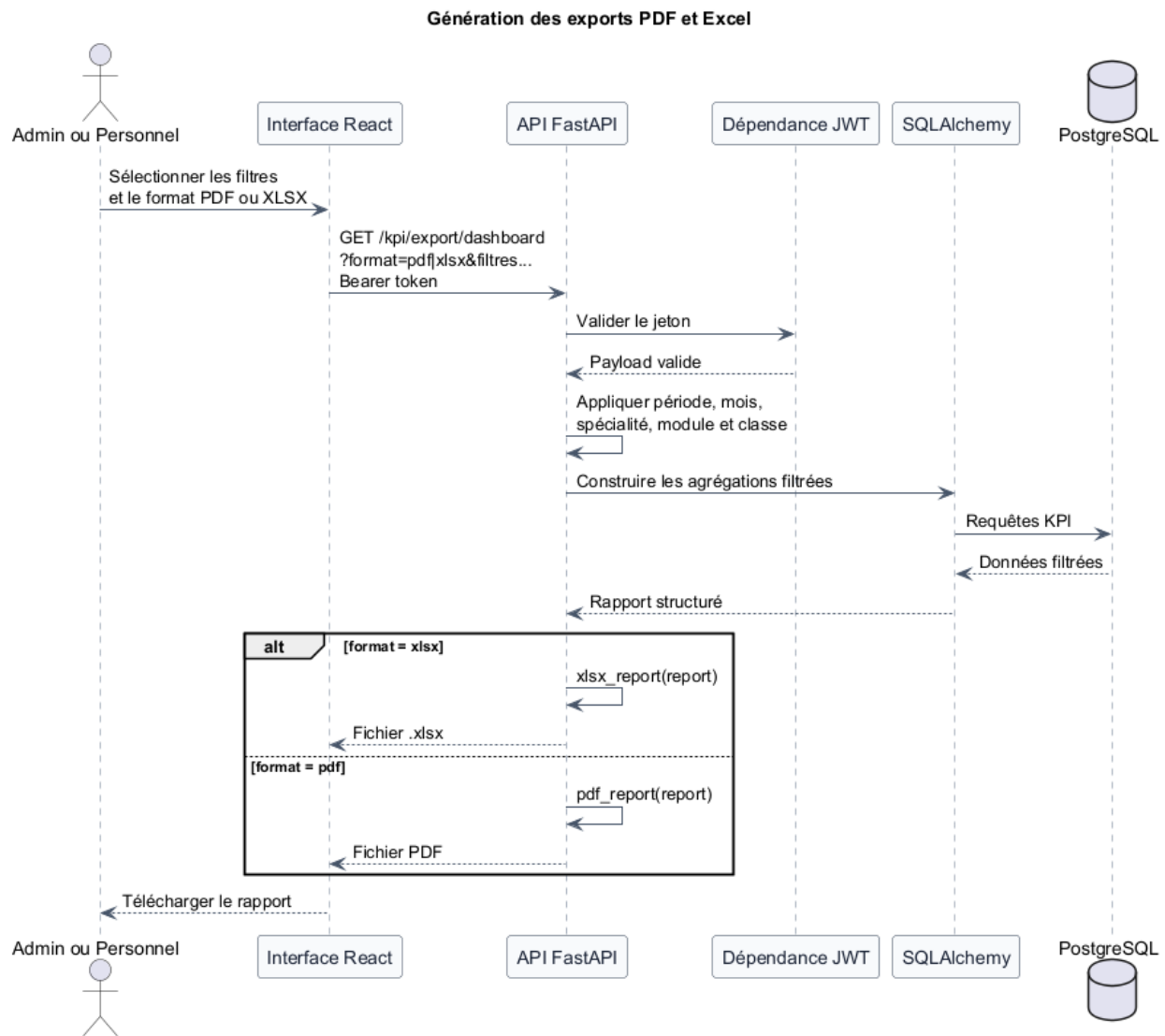


Figure 4 : Séquence de génération des exports PDF et Excel

Cette séquence montre la validation du jeton, l'application des filtres, le calcul des données du rapport et le retour du fichier PDF ou Excel au frontend.

2.11 Synthèse des interactions

Interaction entre :

- Authentification, consultation de la liste des utilisateurs et gestion des comptes : les deux rôles consultent la liste et modifient leur mot de passe ; seul Admin crée ou supprime des comptes.
- Consultation analytique des données étudiantes.
- Analyse des présences et des absences.
- Tableau de bord analytique
- **API Backend**

Les interactions principales relient l'authentification, le service API, les calculs de KPI, les filtres, les exports et la base PostgreSQL.

Chapitre 3 : Conception et architecture

Ce chapitre présente l'architecture générale du système ainsi que son fonctionnement global.

3.1 Architecture générale

Le projet est basé sur une architecture **3-tiers** :

- **Couche présentation (Frontend)** : interface utilisateur développée avec React.
- **Couche métier (Backend)** : gestion des données et logique métier développée en Python.
- **Couche données (Database)** : stockage des données dans PostgreSQL.

3.2 Schéma global du système

Le fonctionnement global du système est le suivant :

Frontend → API Backend → Base de données → Backend → Frontend (cycle logique)

Cette architecture permet une séparation claire des responsabilités et facilite la maintenance et l'évolution du système.

3.3 Avantages de l'architecture

- Modularité du système
- Facilité de maintenance
- Séparation des responsabilités
- Évolutivité du projet
- Sécurité des données

3.4 Conclusion du chapitre

Ce chapitre a présenté l'architecture globale du projet ainsi que son fonctionnement.

L'utilisation d'une architecture 3-tiers garantit un système structuré, performant et facile à maintenir.

Chapitre 4 : Réalisation de l'application

4.1 Développement du backend

L'API FastAPI est initialisée dans `main.py`. Elle inclut les routeurs d'authentification et de KPI, configure CORS et expose la documentation interactive Swagger sur `/docs`.

Ses principales missions sont :

- La création et la gestion des API REST permettant la communication avec le frontend.
- Le traitement des données provenant de la base de données.
- Le calcul des indicateurs de performance (KPI).
- La lecture des données analytiques et les opérations administratives de création ou suppression des comptes utilisateurs.

Le backend joue un rôle essentiel dans la fiabilité et la cohérence des données du système.

4.2 Développement du frontend

Le frontend réutilise un service API commun pour transmettre le jeton JWT et charger les données. Les pages Dashboard et Analytics partagent les mêmes paramètres de filtre, ce qui garantit la cohérence entre cartes KPI, tableaux et graphiques Recharts.

4.3 Captures d'écran à compléter

Les captures suivantes devront être ajoutées manuellement à partir de l'application et du code exécutés localement. Elles ne sont pas simulées dans ce rapport.

La figure suivante présentera `main.py` montrant la création de l'application FastAPI, l'inclusion des routeurs et la configuration CORS.

```
import uvicorn
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import models
from database import engine
from auth.router import router as auth_router
from routes.kpi import router as kpi_router

app = FastAPI()
models.Base.metadata.create_all(bind=engine)
app.include_router(auth_router)
app.include_router(kpi_router)

origins = [
    "http://localhost:5173"
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Figure 5 : Initialisation de l'application FastAPI

La figure suivante présenter auth/router.py montrant les routes login, register, users, delete user et change-password.

```
@router.post("/login")
def login(data: LoginReq, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == data.email).first()

    if not user or not verify_password(data.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid credentials")

    token = create_access_token({
        "sub": user.email,
        "role": user.role,
        "full_name": user.full_name
    })

    return {"access_token": token}

@router.post("/register")
def register(
    data: CreateUserReq,
    db: Session = Depends(get_db),
    user=Depends(require_admin)
):
    existing = db.query(User).filter(User.email == data.email).first()

    if existing:
        raise HTTPException(status_code=400, detail="Email already exists")

    user = User(
        email=data.email,
        full_name=data.full_name,
        password_hash=hash_password(data.password),
        role=data.role
    )

    db.add(user)
    db.commit()
    db.refresh(user)

    return {"message": "User created"}
```

```

@router.get("/users")
def get_users(
    user=Depends(get_current_user),
    db: Session = Depends(get_db)
):
    users = db.query(User).all()

    return [
        {
            "id": u.id,
            "email": u.email,
            "full_name": u.full_name,
            "role": u.role,
            "is_active": u.is_active
        }
        for u in users
    ]

@router.delete("/users/{user_id}")
def delete_user(
    user_id: int,
    current_user=Depends(require_admin),
    db: Session = Depends(get_db)
):
    target_user = db.query(User).filter(User.id == user_id).first()

    if not target_user:
        raise HTTPException(status_code=404, detail="Utilisateur introuvable")

    if target_user.email == current_user.get("sub"):
        raise HTTPException(
            status_code=400,
            detail="Vous ne pouvez pas supprimer votre propre compte"
        )

    if target_user.role == "admin":
        admin_count = db.query(User).filter(User.role == "admin").count()
        if admin_count <= 1:
            raise HTTPException(
                status_code=400,
                detail="Le dernier administrateur ne peut pas être supprimé"
            )

    db.delete(target_user)
    db.commit()

    return {"message": "Utilisateur supprimé avec succès"}

```

```

@router.patch("/change-password")
def change_password(
    data: ChangePasswordReq,
    current_user=Depends(get_current_user),
    db: Session = Depends(get_db)
):
    user = db.query(User).filter(User.email == current_user.get("sub")).first()

    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    if not verify_password(data.old_password, user.password_hash):
        raise HTTPException(status_code=400, detail="Old password is incorrect")

    user.password_hash = hash_password(data.new_password)
    db.commit()

    return {"message": "Password updated"}

```

Figure 6 : Routes d'authentification et de gestion des comptes

Les captures montrent les décorateurs des routes et l'utilisation de `require_admin` sur la création et la suppression.

La figure suivante présenter auth/dependencies.py ou auth_utils.py montrant la validation et le décodage du jeton JWT.

```
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from auth_utils import hash_password, verify_password, create_access_token, decode_token
from fastapi import Depends, HTTPException

security = HTTPBearer()

def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security)):
    token = credentials.credentials
    payload = decode_token(token)

    if payload is None:
        raise HTTPException(status_code=401, detail="Invalid or expired token")

    return payload

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str):
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except JWTError:
        return None
```

Figure 7 : Validation du jeton JWT

La capture montrer HTTPBearer, get_current_user et decode_token.

La figure suivante présenter routes/kpi.py montrant le calcul des indicateurs du tableau de bord.

```
@router.get("/students-count")
def students_count(user=Depends(get_current_user), db: Session = Depends(get_db)):
    return {"students": db.query(Student).count()}

@router.get("/revenue")
def revenue(user=Depends(get_current_user), db: Session = Depends(get_db)):
    return {"revenue": float(db.query(func.sum(Payment.amount)).scalar() or 0)}

@router.get("/enrollments-count")
def enrollments_count(user=Depends(get_current_user), db: Session = Depends(get_db)):
    count = db.query(func.count(distinct(Enrollment.student_id))).scalar() or 0
    return {"enrollments": count}
```

Figure 8 : Calcul des KPI dans le backend

La capture montrer un endpoint KPI et une requête SQLAlchemy utilisée par le dashboard.

La figure suivante présenter models.py montrant les six modèles SQLAlchemy persistés.

```
class Student(Base):
    __tablename__ = "students"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String)
    specialty = Column(String)
    class_name = Column(String)

class Course(Base):
    __tablename__ = "courses"
    id = Column(Integer, primary_key=True)
    title = Column(String)
    specialty = Column(String)

class Enrollment(Base):
    __tablename__ = "enrollments"
    id = Column(Integer, primary_key=True)
    student_id = Column(Integer, ForeignKey("students.id"))
    course_id = Column(Integer, ForeignKey("courses.id"))
    date = Column(Date)

class Payment(Base):
    __tablename__ = "payments"
    id = Column(Integer, primary_key=True)
    student_id = Column(Integer)
    amount = Column(Float)
    date = Column(Date)

class Attendance(Base):
    __tablename__ = "attendance"
    id = Column(Integer, primary_key=True)
    student_id = Column(Integer)
    course_id = Column(Integer)
    status = Column(String)
    date = Column(Date)

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True, index=True)
    full_name = Column(String, nullable=False)
    password_hash = Column(String)
    role = Column(String, nullable=False)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))

    __table_args__ = (
        CheckConstraint(
            "role IN ('admin', 'personnel')",
            name="check_user_role"
        ),
    )
```

Figure 9 : Modèles SQLAlchemy de la base de données

La capture inclure Student, Course, Enrollment, Payment, Attendance et User.

La figure suivante présenter frontend/src/lib/kpi.js montrant l'envoi du jeton Bearer et la construction des filtres.

```
export function authHeaders() {  
  return { Authorization: `Bearer ${localStorage.getItem("token")}` };  
}  
  
export function buildQuery(filters) {  
  const params = new URLSearchParams();  
  Object.entries(filters).forEach(([key, value]) => {  
    if (value) params.set(key, value);  
  });  
  return params.toString();  
}
```

Figure 10 : Service API et transmission du jeton

La capture montrer authHeaders(), buildQuery() et l'URL de l'API.

La figure suivante présenter Dashboard.jsx montrant le chargement des KPI et l'application des filtres.

```
const [filters, setFilters] = useState(emptyFilters);
const [dashboard, setDashboard] = useState(EMPTY_DASHBOARD);
const [loading, setLoading] = useState(true);

const query = useMemo(() => buildQuery(filters), [filters]);

useEffect(() => {
  const loadDashboard = async () => {
    setLoading(true);
    const suffix = query ? `?${query}` : "";
    const response = await fetch(`${API_URL}/kpi/dashboard${suffix}`, {
      headers: authHeaders(),
    });

    if (response.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
      return;
    }

    if (response.ok) setDashboard(await response.json());
    setLoading(false);
  };
  loadDashboard();
}, [query]);

<AcademicFilters
  filters={filters}
  setFilters={setFilters}
  options={options}
/>
```

|

Figure 11 : Chargement des KPI dans le dashboard React

La capture montrer l'appel à /kpi/dashboard et l'utilisation des filtres académiques.

La figure suivante présenter Dashboard.jsx montrant la fonction de téléchargement PDF et Excel.

```
const downloadReport = async (format) => {
  const suffix = query ? `&${query}` : "";
  const response = await fetch(
    `${API_URL}/kpi/export/dashboard?format=${format}${suffix}`,
    { headers: authHeaders() },
  );
  if (!response.ok) return;
  const blob = await response.blob();
  const url = window.URL.createObjectURL(blob);
  const link = document.createElement("a");
  link.href = url;
  link.download = `ofppt-dashboard-report.${format}`;
  link.click();
  window.URL.revokeObjectURL(url);
  setExportOpen(false);
};

<button
  type="button"
  onClick={() => downloadReport("pdf")}
  className="flex w-full items-center gap-2 px-4 py-2 text-sm text-gray-700 hover:bg-gray-50 cursor-pointer"
>
  <FileText size={16} />
  PDF
</button>
<button
  type="button"
  onClick={() => downloadReport("xlsx")}
  className="flex w-full items-center gap-2 px-4 py-2 text-sm text-gray-700 hover:bg-gray-50 cursor-pointer"
>
  <FileSpreadsheet size={16} />
  Excel
</button>
```

Figure 12 : Fonction d'export dans le frontend

La capture montrer l'appel à /kpi/export/dashboard avec les formats pdf et xlsx.

La figure suivante présenter la page de connexion de l'application.

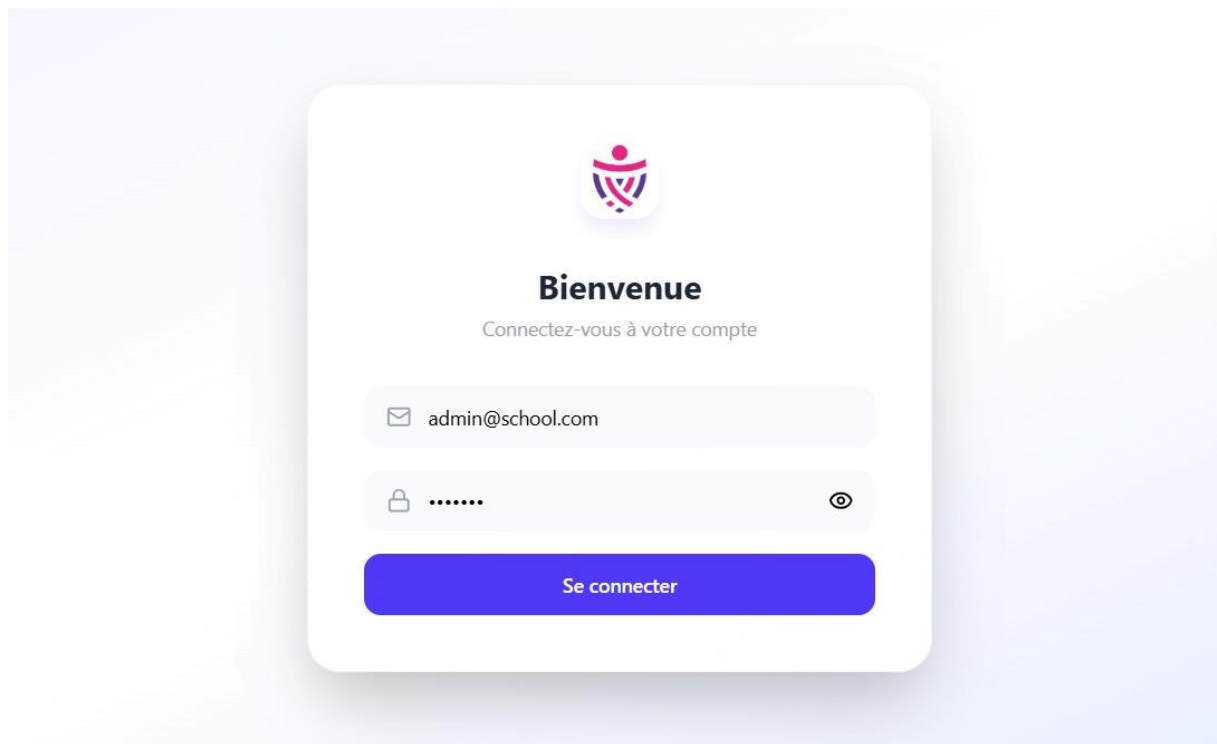


Figure 13 : Page de connexion

La capture montrer le formulaire sans afficher de mot de passe réel.

La figure suivante présenter le dashboard avec les cartes KPI et les filtres.

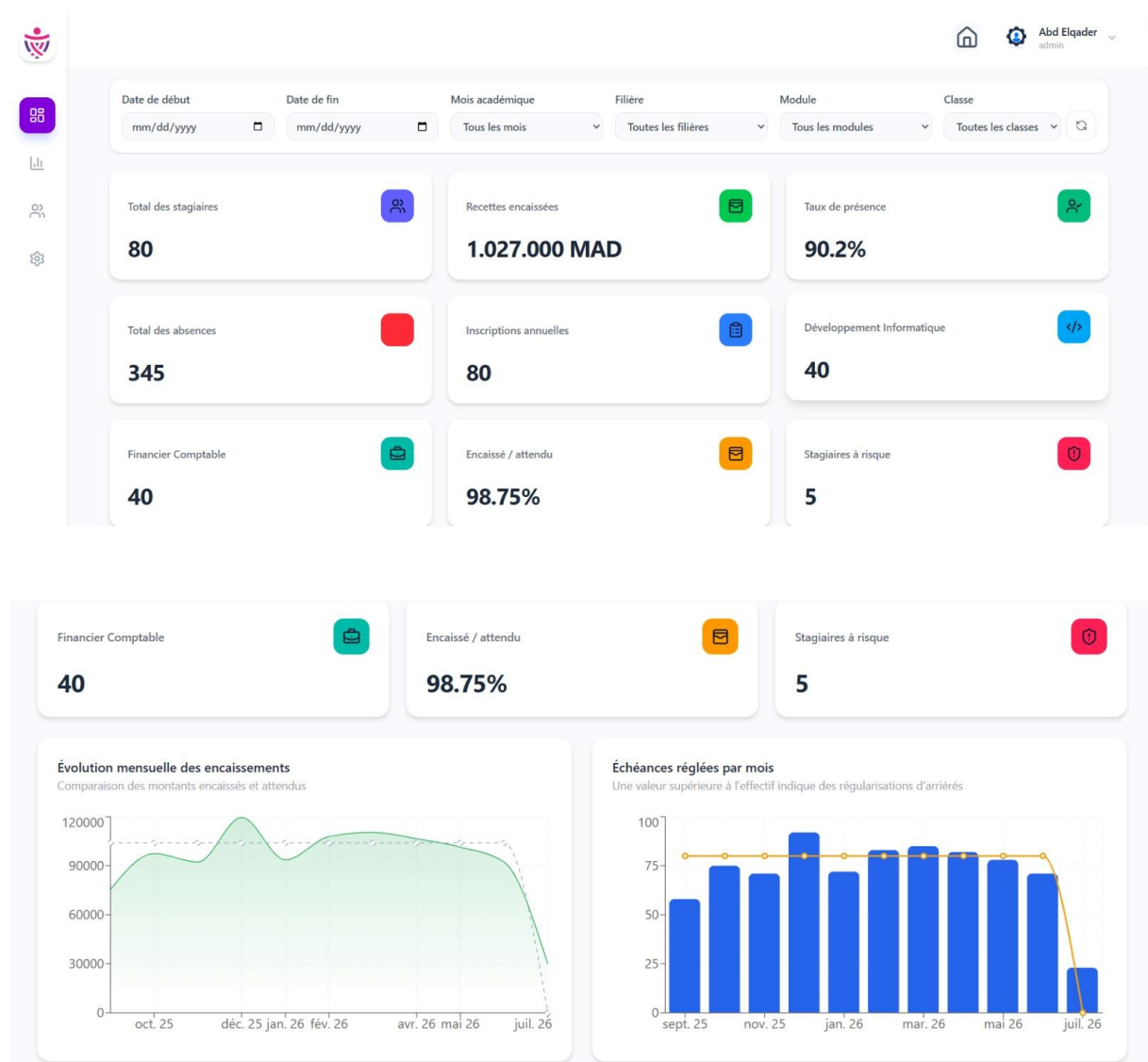


Figure 14 : Tableau de bord analytique

La capture montrer les KPI, les graphiques et les filtres actifs.

La figure suivante, présenter la page Analytics.

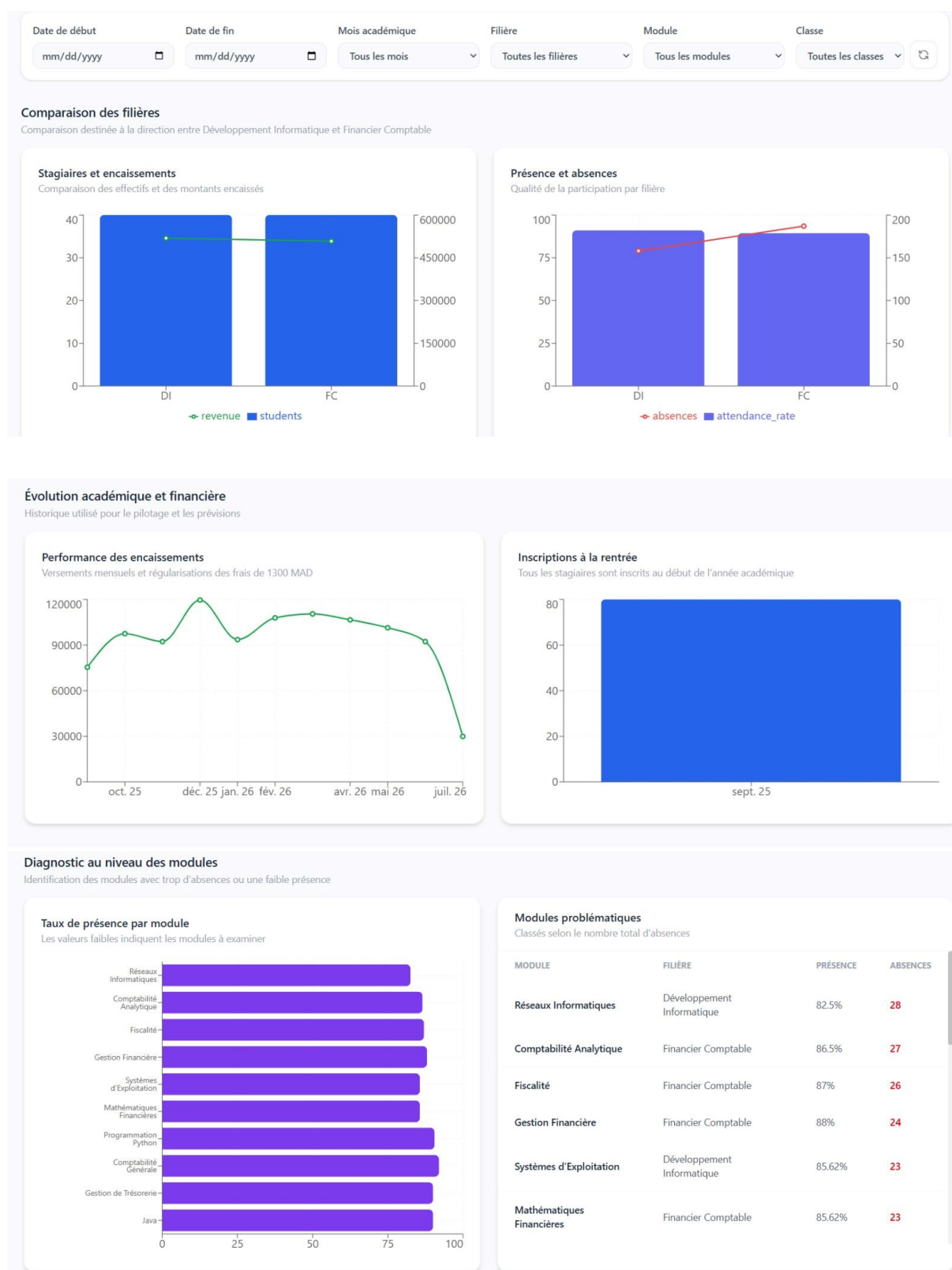


Figure 15 : Page d'analyses comparatives

La capture montrer la comparaison des spécialités et les analyses par module.

La figure suivante présenter la page Utilisateurs pour un compte Admin.

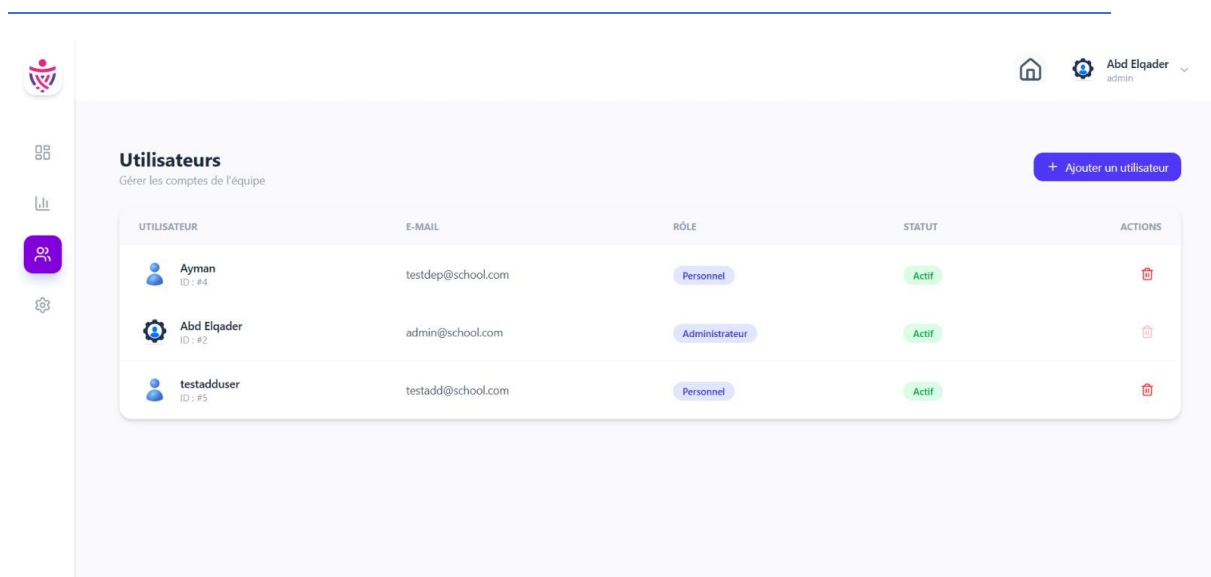


Figure 16 : Gestion des comptes par le rôle Admin

La capture montrer la liste, l'ajout et la suppression ; ces deux actions ne doivent pas être attribuées au rôle Personnel.

La figure suivante présenter la page de changement de mot de passe.

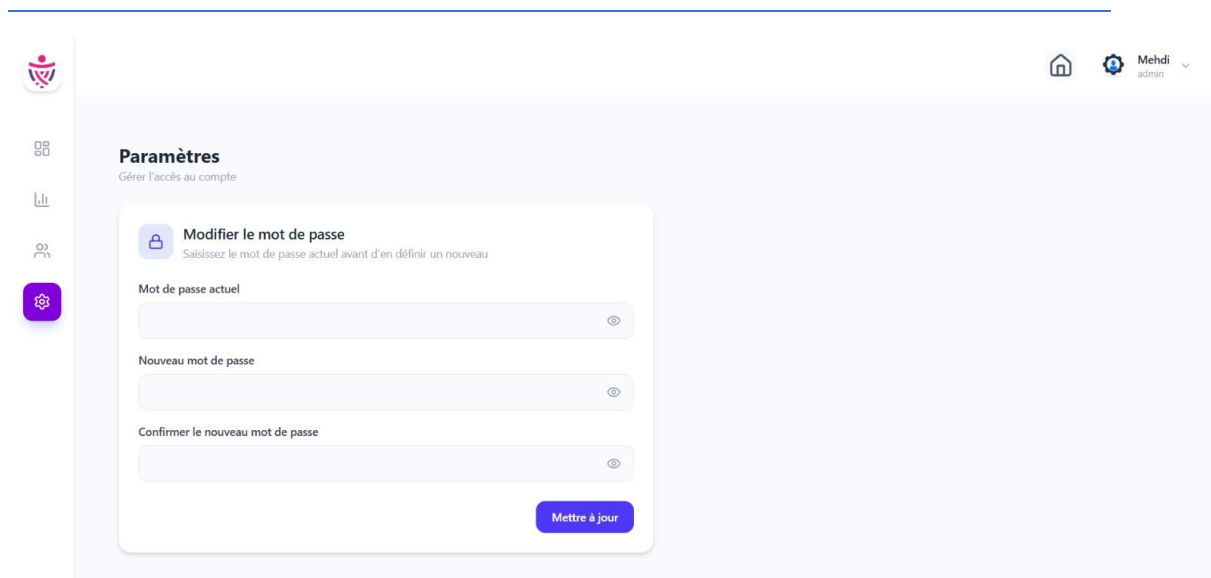


Figure 17 : Modification du mot de passe

La capture montrer l'ancien et le nouveau mot de passe sans valeur sensible.

La figure suivante présenter la documentation Swagger disponible sur /docs.

auth			^
POST	/auth/register	Register	🔒
POST	/auth/login	Login	▼
GET	/auth/users	Get Users	🔒
DELETE	/auth/users/{user_id}	Delete User	🔒
GET	/auth/me	Me	🔒
PATCH	/auth/change-password	Change Password	🔒
kpi			^
GET	/kpi/filters	Filters	🔒
GET	/kpi/dashboard	Dashboard	🔒
GET	/kpi/analytics	Analytics	🔒
GET	/kpi/students-count	Students Count	🔒
GET	/kpi/revenue	Revenue	🔒
GET	/kpi/enrollments-count	Enrollments Count	🔒
GET	/kpi/students-per-course	Students Per Course	🔒
GET	/kpi/attendance-rate	Attendance Rate	🔒
GET	/kpi/absences	Absences	🔒
GET	/kpi/revenue/monthly	Monthly Revenue	🔒
GET	/kpi/students-growth	Students Growth	🔒
GET	/kpi/attendance-summary	Attendance Summary	🔒
GET	/kpi/recent-payments	Recent Payments	🔒
GET	/kpi/courses	Courses	🔒
GET	/kpi/trends	Trends	🔒
GET	/kpi/course-popularity	Course Popularity	🔒
GET	/kpi/repeated-absences	Repeated Absences	🔒
GET	/kpi/payments/monthly-summary	Monthly Payment Summary	🔒
GET	/kpi/ai-insights	AI Insights	🔒
GET	/kpi/export/dashboard	Export Dashboard	🔒

Figure 18 : Documentation interactive FastAPI

La figure suivante présenter deux fichiers réellement générés, l'un au format PDF et l'autre au format Excel.

Rapport du Tableau de Bord de Direction

Institut de Formation - Technicien Spécialisé

Filtres appliqués

Filtre	Valeur
Start Date	Tous
End Date	Tous
Academic Month	Tous
Specialty	Tous
Module ID	Tous
Class Name	Tous

Indicateurs clés (KPI)

Indicateur	Valeur
Stagiaires	80
Recettes encaissées	1 027 000 MAD
Recettes attendues	1 040 000 MAD
Taux de recouvrement	98,75 %
Taux de présence	90,2 %
Absences	345
Inscriptions annuelles	80
Stagiaires à risque	5

Répartition par filière

Filière	Stagiaires	Recettes	Présence
Développement Informatique	40	518 700 MAD	91,02 %
Financier Comptable	40	508 300 MAD	89,38 %

Évolution mensuelle des encaissements

Mois	Montant (MAD)
2025-09	75 400
2025-10	97 500
2025-11	92 300
2025-12	119 600
2026-01	93 600

2026-02	107 900
2026-03	110 500
2026-04	106 600
2026-05	101 400
2026-06	92 300
2026-07	29 900

Modules avec le plus d'absences

Module	Absences
Réseaux Informatiques	28
Comptabilité Analytique	27
Fiscalité	26
Gestion Financière	24
Systèmes d'Exploitation	23

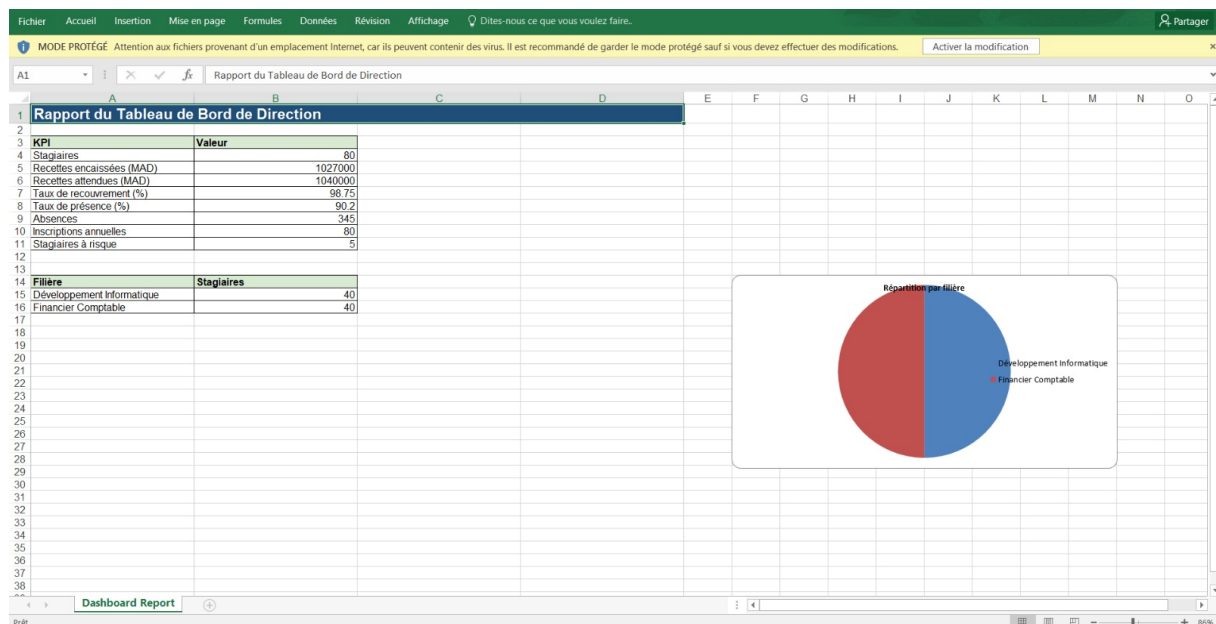


Figure 19 : Résultat des exports PDF et Excel

La capture montre les fichiers ouverts et les filtres utilisés pour leur génération.

4.4 Modélisation de la base de données

PostgreSQL stocke les données métier et les comptes. La connexion est configurée par l'URL de base de données, tandis que SQLAlchemy définit les modèles et exécute les requêtes.

La base PostgreSQL contient six entités principales : Student, Course, Enrollment, Payment, Attendance et User.

- Students : identité, spécialité et classe de chaque étudiant.
- Courses : modules rattachés à Développement Informatique ou Financier Comptable.
- Enrollments : inscriptions pédagogiques reliant Student et Course par deux clés étrangères.

Payments : règlements datés associés logiquement aux étudiants.

Attendance : présence ou absence par étudiant, module et date.

Users : comptes applicatifs avec rôle admin ou personnel.

Les relations entre ces tables permettent une exploitation efficace des données et facilitent les analyses statistiques.

4.5 Fonctionnalités principales

Le système implémente plusieurs fonctionnalités essentielles :

- Consultation analytique des étudiants, classes, modules, inscriptions, présences et paiements.
- Comparaison des spécialités et suivi des inscriptions annuelles.
- Visualisation filtrable des tendances et comparaisons avec Recharts.
- Calcul des KPI d'effectif, assiduité, absences, recettes et recouvrement.
- Export du rapport filtré aux formats PDF et Excel (.xlsx).

Ces fonctionnalités permettent d'offrir une vision globale et structurée des données de l'établissement.

4.6 Sécurité du système

La sécurité est un élément fondamental dans ce projet. Plusieurs mesures ont été mises en place :

- Authentification des utilisateurs via login et mot de passe.
- Protection des API contre les accès non autorisés.
- Validation des données envoyées par le frontend.
- Sécurisation de la base de données et des requêtes.
- Ces mesures garantissent l'intégrité et la confidentialité des données

4.7 Performance et optimisation

Afin d'assurer un bon fonctionnement du système, plusieurs optimisations ont été appliquées :

- Optimisation des requêtes vers la base de données.
- Amélioration des performances du frontend pour un affichage rapide des graphiques.

Ces optimisations permettent d'améliorer la rapidité et la fluidité de l'application.

4.8 Déploiement local et perspectives de mise en production

Ce chapitre présente les différentes étapes liées au déploiement de l'application ainsi que les techniques utilisées pour optimiser ses performances et garantir son bon fonctionnement dans un environnement réel.

4.8.1 État actuel de l'hébergement

À la date de finalisation du rapport, l'application est uniquement testée en local. Aucune URL publique de production n'est disponible.

- Environnement local : frontend React sur localhost:5173 et API FastAPI sur localhost:8000.
- Environnement de production : non déployé à la date de rédaction.

Le frontend React pourra être déployé sur Netlify. L'API FastAPI devra être hébergée séparément sur Render, Railway ou un VPS, avec un service PostgreSQL accessible.

4.8.2 Étapes de mise en production

Le déploiement de l'application suit plusieurs étapes essentielles :

- Préparation du backend et vérification des API.
- Construction et optimisation du frontend.
- Configuration de la base de données PostgreSQL.
- Connexion entre le frontend, le backend et la base de données.
- Mise en ligne de l'application sur un serveur d'hébergement.
- Tests finaux pour vérifier le bon fonctionnement global du système.

Ces étapes garantissent un déploiement stable et fonctionnel.

4.8.3 Solutions d'hébergement envisagées

Pour le déploiement, plusieurs outils peuvent être utilisés :

- Serveur local (localhost) pour les tests.
- Frontend envisagé sur Netlify ; backend FastAPI sur Render, Railway ou un VPS.
- Outils de gestion de version comme Git et GitHub.
- Environnements de production configurés pour exécuter le backend et le frontend.

4.8.4 Optimisation

L'optimisation vise à améliorer la performance et la rapidité de l'application.

Les principales techniques utilisées sont :

- Optimisation des requêtes SQL pour réduire le temps de réponse.
- Réduction des appels API inutiles entre frontend et backend.
- Optimisation des composants frontend pour améliorer le chargement des pages.
- Structuration efficace du code pour faciliter la maintenance.

4.8.5 Configuration de production

La sécurité est renforcée lors du déploiement par :

- Utilisation de connexions sécurisées (HTTPS).
- Protection des API contre les accès non autorisés.
- Gestion des authentifications et des tokens.
- Sécurisation de la base de données et des accès serveur.

La mise en production exigera la configuration de l'URL de base de l'API, des origines CORS, de DATABASE_URL, du secret JWT et de HTTPS. Ces paramètres devront être stockés dans des variables d'environnement et non dans le code source.

4.9 Conclusion du chapitre

Ce chapitre a présenté la réalisation du backend et du frontend, la sécurité, les exports et les conditions d'un futur déploiement. L'application demeure actuellement exécutée et validée dans un environnement local.

Chapitre 5 : Tests et résultats

5.1 Scénarios de validation

Les scénarios suivants vérifient les fonctions principales observées dans le code et lors des essais locaux. Les validations visuelles finales restent à documenter par des captures d'écran.

Tableau 2 : Scénarios de validation fonctionnelle

Test	Objectif	Résultat attendu	Résultat obtenu	Statut
Connexion valide	Authentifier un compte existant	JWT retourné et accès à l'application	Fonction présente dans /auth/login	À confirmer avec capture finale
Connexion invalide	Refuser un mot de passe incorrect	Réponse 401 sans jeton	Contrôle verify_password présent	À confirmer avec capture finale
Route sans token	Vérifier la protection JWT	Accès refusé	HTTPBearer et get_current_user appliqués	Conforme au code
Chargement des KPI	Afficher le dashboard	Cartes et graphiques alimentés	Route /kpi/dashboard et appels React présents	À confirmer avec capture finale
Application des filtres	Filtrer les indicateurs	Résultats recalculés selon les filtres	Paramètres date, mois, spécialité, module et classe implémentés	À confirmer avec capture finale
Export PDF	Générer le rapport filtré	Fichier PDF téléchargeable	Route et générateur PDF présents	À confirmer avec capture finale
Export Excel	Générer le classeur filtré	Fichier .xlsx téléchargeable	Route et générateur XLSX présents	À confirmer avec capture finale
Liste des utilisateurs	Vérifier l'accès Admin et Personnel	Les deux rôles obtiennent la liste	/auth/users utilise get_current_user	Conforme au code
Création par Admin	Créer un compte utilisateur	Création autorisée	/auth/register utilise require_admin	À confirmer avec capture finale
Suppression par Admin	Supprimer un autre compte	Suppression autorisée selon les règles	DELETE /auth/users/{user_id} utilise require_admin	À confirmer avec capture finale
Restrictions Personnel	Interdire création et suppression	Réponse 403	require_admin vérifie role == admin	Conforme au code
Auto-suppression	Protéger le compte courant	Suppression refusée	Comparaison avec current_user.sub implémentée	Conforme au code
Dernier Admin	Conserver au moins un Admin	Suppression refusée	Comptage des comptes admin implémenté	Conforme au code
Changement de mot de passe	Vérifier l'ancien mot de passe	Modification seulement si l'ancien est correct	Route /auth/change-password présente	À confirmer avec capture finale
Build frontend	Valider la compilation React/Vite	Build sans erreur bloquante	Build exécuté dans le cadre de la revue	Conforme

5.2 Résultats obtenus

Synthèse des réalisations

Les données simulées de démonstration couvrent septembre 2025 à juillet 2026. Le jeu de test contient 80 étudiants fictifs répartis dans quatre classes, 20 modules, des inscriptions simulées en septembre 2025, des présences par module et des paiements irréguliers destinés à illustrer les retards et règlements cumulés.

Les principales réalisations sont les suivantes :

- Consultation analytique des étudiants, inscriptions, paiements et présences, sans CRUD étudiant dédié.
- Base PostgreSQL structurée autour des étudiants, modules, inscriptions, paiements, présences et utilisateurs.

La prévision utilise une régression linéaire, choisie pour sa simplicité et son interprétation directe. Les anomalies sont signalées par écart statistique ; les étudiants à risque sont identifiés par un seuil d'absences ; la segmentation K-Means regroupe les profils d'assiduité et de paiement.

- Visualisation des données filtrées dans une interface responsive.
- Export des rapports aux formats PDF et Excel.

Ce projet a permis de consolider plusieurs compétences techniques : développement web (frontend et backend), conception d'API REST, modélisation de bases de données, visualisation de données et gestion de projet.

5.3 Difficultés rencontrées et solutions apportées

Tableau 3 :difficultés et solutions

Difficulté	Solution
Communication entre frontend et backend	Mise en place d'API REST avec FastAPI
Gestion et structuration des données	Conception d'un modèle relationnel avec PostgreSQL
Affichage des graphiques dynamiques	Utilisation de Recharts pour les graphiques et de Lucide React pour les icônes.
Gestion des KPI et statistiques	Traitement des données côté backend

5.4 Limites et améliorations possibles

Améliorations futures

Les limites actuelles sont l'absence de données de satisfaction et de retard horaire, l'absence de pages métier CRUD pour les étudiants, présences et paiements, ainsi que l'absence de déploiement public. Ces éléments ne sont donc pas présentés comme réalisés.

Perspectives d'évolution

- À moyen et long terme, le Dashboard Analytique pourrait évoluer vers une plateforme Intelligente et complète

- Les analyses intelligentes sont déjà intégrées : tendance linéaire des paiements, anomalies statistiques, étudiants à risque et segmentation K-Means.
- Développement d'une application mobile (React Native ou Flutter).
- Extension du système pour gérer plusieurs établissements scolaires.
- Ajout d'un système avancé de data visualisation en temps réel.
- Amélioration continue de la sécurité et des performances du système.

Conclusion générale

Ce projet a permis de réaliser un dashboard analytique cohérent avec le fonctionnement d'une école marocaine de techniciens spécialisés. La solution exploite FastAPI, SQLAlchemy, PostgreSQL, React et Recharts pour présenter des indicateurs fiables sur les effectifs, l'assiduité, les absences et le recouvrement.

Les filtres globaux, les comparaisons entre Développement Informatique et Financier Comptable, les exports PDF/Excel et les méthodes analytiques explicables apportent une aide concrète à la décision.

Cependant, durant la réalisation du projet, plusieurs difficultés ont été rencontrées, notamment :

- la synchronisation entre le frontend et le backend via les API,
- la gestion et la structuration des données dans la base de données,
- ainsi que l'optimisation des performances lors de l'affichage des statistiques.

Ces problèmes ont été résolus progressivement grâce à la recherche de solutions techniques et à l'amélioration continue du code.

Les prochaines étapes concernent le déploiement public, l'ajout éventuel de données de satisfaction et de retard horaire, l'enrichissement des tests automatisés et l'intégration de nouvelles années de démonstration pour consolider les prévisions.

- La prévision des tendances, la détection d'anomalies et la segmentation sont déjà intégrées sous une forme légère et explicable.

En conclusion, ce projet nous a permis de renforcer nos compétences en développement web, en gestion de bases de données et en analyse de données, tout en proposant une solution concrète et utile pour la gestion des établissements scolaires.

Bibliographie & Webographie générale

Bibliographie

Silberschatz, A., Korth, H. F. et Sudarshan, S., Database System Concepts, McGraw-Hill.

Webographie

FastAPI — Documentation officielle : <https://fastapi.tiangolo.com/>

React — Documentation officielle : <https://react.dev/>

PostgreSQL — Documentation officielle : <https://www.postgresql.org/docs/>

SQLAlchemy — Documentation officielle : <https://docs.sqlalchemy.org/>

Vite — Documentation officielle : <https://vite.dev/guide/>

Recharts — Documentation officielle : <https://recharts.org/>

Netlify — Documentation officielle : <https://docs.netlify.com/>

Annexes

Les annexes regroupent les éléments complémentaires utilisés pour la conception, le développement et la validation de l'application. Elles permettent d'illustrer les principaux aspects techniques du projet et de fournir des preuves visuelles des fonctionnalités réalisées.

Annexe A : Captures de l'application

Cette annexe contient les captures d'écran finales de l'application, notamment :

- * La page de connexion.
- * Le tableau de bord principal et ses indicateurs clés (KPI).
- * Les filtres académiques.
- * La gestion des utilisateurs.
- * La modification du mot de passe.
- * Les exports PDF et Excel.
- * Les différentes vues analytiques disponibles.

Annexe B : Diagrammes UML

Cette annexe présente les diagrammes UML actualisés du projet :

- * Diagramme de cas d'utilisation.
- * Diagramme de classes.
- * Diagramme de séquence (authentification et chargement des KPI).
- * Autres diagrammes techniques si nécessaire.

Annexe C : Schéma de la base de données PostgreSQL

Cette annexe présente le schéma relationnel de la base de données PostgreSQL, incluant les principales entités du système :

- * User
- * Student
- * Course
- * Enrollment

- * Payment

- * Attendance

Elle met également en évidence les relations entre les différentes tables utilisées pour le calcul des indicateurs du tableau de bord.

Annexe D : Documentation Swagger

Cette annexe contient un extrait de la documentation interactive FastAPI accessible via l'URL /docs.

Les principaux groupes de routes exposés par l'API sont :

- * /auth : authentification, profil utilisateur, gestion des utilisateurs et changement de mot de passe.

- * /kpi : indicateurs du tableau de bord, filtres académiques, tendances, comparaisons, analyses et exports PDF/Excel.